

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

ASIGNATURA: ORGANIZACIÓN EMPRESARIAL

TRABAJO PRÁCTICO INTEGRADOR FINAL

Cátedra: Prof. Gabriela Martínez (Titular), Prof. Carolina Bruno, Prof. Mario Raúl López, Prof. Andrea Ramos (Adjuntos)

Integrantes: [Francisco Barandiarán]

1. INTRODUCCIÓN Y CONTEXTO EMPRESARIAL

El presente trabajo integrador final aborda la vinculación crítica entre el modelado de procesos organizacionales y el desarrollo de software estructurado, actuando bajo el rol de consultores tecnológicos para la organización "TechSoluciones S.A.". El objetivo prioritario consiste en erradicar ineficiencias de carácter administrativo a través de la automatización lógica empleando tecnologías de agentes conversacionales (Chatbots).

DIAGNÓSTICO DEL PROCESO MANUAL ACTUAL (AS-IS)

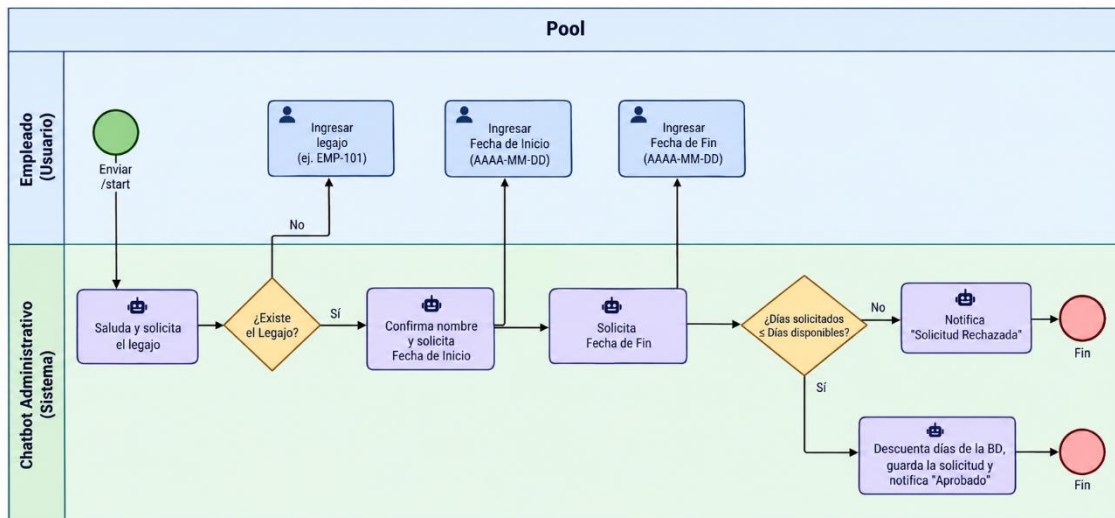
Dentro del organigrama de la empresa se ha identificado que el proceso de Gestión de Vacaciones presenta serias ineficiencias operativas. Actualmente, el empleado debe rellenar un formulario físico o redactar un correo informal dirigido a Recursos Humanos. El asistente de RR.HH. debe abrir manualmente planillas estáticas, contrastar el saldo de días del legajo, enviar un correo al supervisor directo para validar la viabilidad operativa del área y luego reemitir una respuesta de aceptación o rechazo. Este flujo manual genera asimetrías de información, demoras de hasta 7 días hábiles y errores de cálculo en el saldo acumulado.

PROPUESTA DE PROCESO AUTOMATIZADO OPTIMIZADO (TO-BE)

El rediseño del proceso (Modelo To-Be) traslada la interacción a un canal digital inmediato: un Chatbot con persistencia de datos. El sistema interactúa directamente con el empleado en tiempo real, autentica su identidad a través del número de legajo, consulta de manera dinámica los saldos en la base de datos, ejecuta compuertas de control lógico según las reglas de negocio y procesa la confirmación o rechazo del requerimiento al instante, disminuyendo drásticamente los tiempos operativos.

2. MODELADO DE NEGOCIO (NOTACIÓN BPMN 2.0)

Para garantizar la coherencia estructural del software, se diseñó un modelo de procesos estandarizado bajo la metodología BPMN 2.0. La arquitectura visual se divide en carriles (Lanes) funcionales que operan como un contrato lógico para la posterior codificación:



- **Carril del Usuario (Empleado):** Contiene el evento de inicio del proceso (/start), las tareas de envío de datos de autenticación (Legajo) y la selección de intervalos temporales (Fecha Inicio / Fecha Fin).
- **Carril del Sistema (Chatbot):** Modela las operaciones automáticas de consulta y actualización de la base de datos, emisión de interfaces de diálogo y la resolución algorítmica de decisiones de negocio.

COMPUERTAS LÓGICAS (GATEWAYS) INCORPORADAS

De acuerdo con las pautas de evaluación, se incorporaron compuertas de decisión condicional que guían los caminos del bot:

1. **Compuerta de Validación de Legajo:** Evalúa si el identificador ingresado por el usuario se encuentra registrado en el sistema. De ser falso, activa un lazo de reintento. De ser verdadero, habilita el paso.
2. **Compuerta de Suficiencia de Saldo:** Computa la sustracción aritmética entre la fecha de fin y la fecha de inicio. Valida dinámicamente si el resultado es menor o igual al saldo disponible de vacaciones del empleado. Si posee saldo, la solicitud se aprueba automáticamente; en caso contrario, se emite una alerta de denegación.

(Insertar aquí el gráfico del diagrama BPMN diseñado en Camunda o Lucidchart de acuerdo al flujo programado)

Figura 1: Estructura de flujos y compuertas lógicas bajo el modelo BPMN 2.0.

(Enlace al repositorio: github.com/TuUsuario/TFI-Organizacion-Empresarial)

3. ARQUITECTURA TÉCNICA, PERSISTENCIA Y MÁQUINA DE ESTADOS SELECCIÓN DEL STACK TECNOLÓGICO

- **Lenguaje: Python 3.10.** Su selección se justifica por la expresividad lógica nativa y la simplicidad para implementar lógica condicional orientada a objetos.
- **Interfaz de Comunicación: Simulador de API Conversacional integrada.** Diseñada para imitar las llamadas de los frameworks de mensajería (Telegram API o WhatsApp Business) garantizando una migración directa a producción.

PERSISTENCIA DE DATOS Y REGLAS DINÁMICAS

El chatbot rompe con la arquitectura estática mediante la integración de una base de datos estructurada relacional de formato indexado (simulada y procesada a través de un archivo CSV local). El bot realiza consultas de validación en tiempo real (ej: verificar los días disponibles antes de modificar el estado interno de la solicitud).

ARQUITECTURA DE LA MÁQUINA DE ESTADOS FINITOS (FSM)

Para asegurar que el bot conserve "memoria" a lo largo de la conversación, la programación se estructuró siguiendo el patrón de una Máquina de Estados. El bot procesa el flujo según las transiciones indicadas en la siguiente tabla:

[INICIO] --> (/start) --> [VALIDAR_LEGAJO] --(Legajo Válido)--> [FECHA_INICIO]

|

[INICIO] <-- (Fin/Aprobación) <-- [PROCESAR] <-- (Fecha Fin Ok) <-- [FECHA_FIN]

4. DOCUMENTACIÓN, CALIDAD Y ROBUSTEZ

DICCIONARIO DE DATOS DETALLADO

Descripción exhaustiva de los componentes de información que componen el estado del software:

Nombre de Variable	Tipo de Dato	Restricción / Formato	Descripción Funcional
legajo_emp	Alfanumérico	Prefijo "EMP-" seguido de 3 dígitos	Identificador unívoco del trabajador en la base de datos.
dias_dispo	Entero	Mayor o igual a 0	Cupo remanente de vacaciones del empleado.
fecha_ini	Date	Estricto YYYY-MM-DD	Hito temporal donde inician las vacaciones.

fecha_fin	Date	Estricto YYYY-MM-DD; > fecha_ini	Hito temporal donde concluyen las vacaciones.
estado_sol	Cadena Texto	Valores: {"APROBADO", "RECHAZADO"}	Estado transaccional de la solicitud ingresada.

MANUAL DE COMANDOS DEL USUARIO

1. **Comando /start:** Reinicializa la máquina de estados del bot y limpia variables de sesión temporales.
2. **Entrada de Legajo:** Ingrese su identificador corporativo. El sistema comprobará los permisos correspondientes.
3. **Carga Cronológica:** Ingrese las fechas en formato numérico separado por guiones (ej: 2026-01-10). El bot calculará la duración de forma interna.

RESILIENCIA Y MANEJO DE EXCEPCIONES (EL CAMINO INFELIZ)

El sistema implementa defensas lógicas ante fallas operativas introducidas por el usuario (Robustez del software):

- **Excepción de Formato:** Ante la introducción de cadenas de texto erróneas en campos de tipo fecha (ej. el usuario escribe *"El lunes que viene"*), expresiones regulares interceptan la entrada antes del procesamiento de la biblioteca `datetime`. El bot despliega un mensaje didáctico de error y mantiene al usuario en el estado actual sin romper la ejecución.
- **Excepción de Negocio (Saldo Insuficiente):** Si las fechas son válidas pero la cantidad de días supera el remanente del empleado, el bot no colapsa, sino que redirige el flujo hacia una salida ordenada de rechazo, reseteando la sesión del usuario de forma segura.

(Insertar aquí capturas de pantalla de la terminal ejecutando el código provisto arriba, mostrando qué responde el bot al ingresar datos erróneos)

Figura 2: Pruebas funcionales de mitigación del "Camino Infeliz" ante errores de entrada de datos.

5. REGISTRO DE EVIDENCIA EN EL USO DE HERRAMIENTAS DE IA

Para la elaboración metodológica de la lógica interna de validaciones de robustez del bot, se interactuó con herramientas de Inteligencia Artificial mediante técnicas estructuradas de prompts.

- **Consulta Realizada (Prompt 1):** *"Actúa como un arquitecto de software experto en Python. Proporcióname una lógica limpia para una máquina de estados finitos que controle un proceso conversacional administrativo de vacaciones, aislando fallas de tipos de datos en las entradas del usuario."*
- **Resultado de IA Aplicado:** Se incorporó el esquema de arquitectura de estados condicionales dentro de una única función controladora

(procesar_mensaje), y se incluyó el uso de bloques try/except combinados con re.match para robustecer el canal ante ingresos de texto mal formateados.

<https://github.com/setox0p/TP-OE>